
Exame de Estruturas de Dados – 2010.Julho.13

Duração: 2:30 horas

Valorização: 12 valores.

1. Responda às seguintes questões:

- a) Para um dado problema com tamanho de input n , executamos os algoritmos A, B e C. Em termos de tempo de execução, cada um dos algoritmos assume uma das seguintes ordens de complexidade $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, e $\mathcal{O}(n^2)$. Alguns dos tempos medidos para estes algoritmos são evidenciados na tabela seguinte:

		Input		
		512	1024	2048
Algoritmo	A	70	134	262
	B	135	517	2053
	C	42	86	182

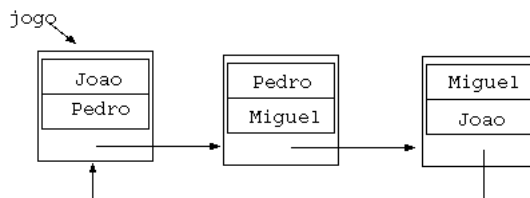
Estabeleça e explique a correspondência entre o algoritmo e a ordem de complexidade que estará associada em face dos tempos apresentados.

- b) Coloque por ordem crescente de grandeza de complexidade as medidas $\mathcal{O}(n^2/100)$, $\mathcal{O}(500 \log n)$, $\mathcal{O}(n \log 3n)$, $\mathcal{O}(50^5)$.
- c) Considere ainda a sequência 12, 6, 9, 17, 20, 14, 6. Desenhe as transformações sucessivas que serão necessárias para transformar a sequência numa árvore de prioridade em que o valor na raiz é o valor de prioridade máxima.

2. Listas circulares

O jogo *Assassin* é um jogo de vida-real com um grupo de jogadores em que todos tentam, individualmente, encontrar e anular um outro jogador (o *paintball* será uma variante deste jogo). A cada jogador é atribuído aleatoriamente o nome do jogador que tem de anular (a sua vítima). Quando um jogador anula uma vítima, o jogador que era o alvo desta vítima passa a ser o próximo alvo do jogador bem sucedido. O jogo continua até que sobre apenas um jogador e este será declarado o vencedor.

Suponha que lhe pedem para simular este jogo. Após a geração aleatória dos nomes dos jogadores alvos de cada jogador, estabelece-se uma cadeia circular entre os nomes dos jogadores, como se ilustra na figura seguinte:



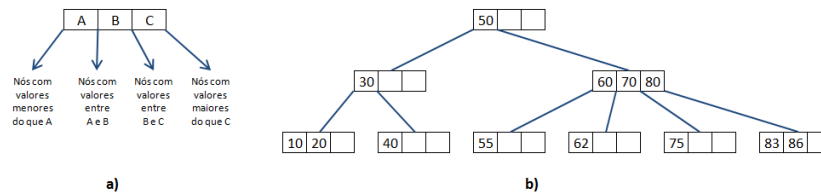
- a) Implemente as estruturas de dados necessárias para representar um jogador e uma classe `JogoAssassin` com um atributo `jogo` que representa a lista circular que se estabelece entre os jogadores. A lista circular deverá ser uma lista ligada simples. Escreva os métodos construtores de objectos e um método `procura(nome)` em que dado o `nome` de um jogador, retorna uma referência para o nó que representa o jogador com esse `nome`.
- b) Implemente na classe `JogoAssassin` um método `kill(nome)` que anule o jogador com o nome `nome`. Repare que quando anula o jogador X que tinha como alvo o jogador Y, tem de procurar o jogador Z que tinha como alvo X para poder actualizar o alvo de Z que passará a ser Y.

No exemplo da figura, se removesse o jogador João, teria de actualizar o alvo do Miguel para passar a ser o Pedro em vez do João.

- c) Implemente na classe `JogoAssassin` um método `simula()` que faz a simulação do jogo e retorna o nome do jogador vencedor. Para simular o jogo, considere que existe um método `String nextVictim()` que quando invocado retorna o nome do jogador que será a próxima vítima, i.e. o próximo jogador a anular. A simulação termina quando apenas sobrar um jogador na lista circular.

3. Árvores

Para além das árvores binárias, existe um outro tipo de árvores, as **árvores 2-3-4** ou **árvores 2-4**, muito utilizadas, por exemplo, para implementar dicionários. Este tipo de árvore permite o armazenamento de, até, 3 valores por nó, e têm dois, ou três, ou quatro nós filhos (daí o seu nome). Nas árvores 2-3-4 os valores nos nós vão sendo preenchidos da esquerda para a direita; os valores vão sendo introduzidos de forma ordenada na árvore, e um nó, ou tem zero filhos, ou tem 2, ou 3, ou 4 filhos. Esta situação é exemplificada conceptualmente na figura 1a) e através de um exemplo concreto na figura 1b):



Considere as seguintes classes que implementam um nó e a própria árvore:

```
class Node {
    private static final int ORDER= 3;
    private int nItems;
    private Node childArray[]= new Node[ORDER+1];
    private int itemArray[]= new int[ORDER];
    // ... métodos...
    public boolean isLeaf() {return childArray[0]==null;}
    public int getVal(int pos) {return itemArray[pos];}
    public Node getChildNode(int pos) {return childArray[pos];}
    public int orderPos(int val) {...};
    public void addItem(int val, int pos) {...};
}

class Tree234 {
    private Node root;
    // ... métodos ...
}
```

- a) A inserção de valores nas árvores 2-3-4 acontece sempre nos nós-folha. Quando estes estão cheios, há necessidade de se fazer um *split* e propagar valores para os nós intermédios ou mesmo criar um novo nível. Crie o método `public void insertVal(int novoVal, Node r)` da classe `Tree234` para inserir um novo valor na árvore a partir do nó `r`. Pode assumir que os nós-folha não estarão totalmente preenchidos, pelo que não precisará de se preocupar com a implementação da operação *split*.

Sugestão: faça uso dos já definidos e implemente os métodos `orderPos()` e `addItem()`. `orderPos(int val)` deverá retornar a posição `j` do vector `itemArray[]` em que o valor `val` é maior ou igual que `itemArray[j-1]` e menor que o seguinte. O método `addItem(int val, int pos)` adiciona o novo valor na posição `pos` do vector `itemArray[]` libertando previamente espaço.

- b) Na classe `Tree234`, crie o método `public int find(int val, Node r)` que procura o valor `val` nos nós da árvore, iniciando a procura no nó `r` e retorna o índice da posição do nó em que se encontre o valor a procurar, ou -1 se esse valor não existir na árvore.

Sugestão: use os métodos da classe `Node` já implementados.